

Step 1: Study the given Main.java carefully.

```
// University of Karachi - Department of Computer Science (DCS)
// Simple Student Registration Story using OOP and custom StudentList
```

```
// DCS_Project/
//   └─ Student.java
//   └─ StudentList.java
//   └─ Main.java
```

```
public class StudentList {
    private Student[] students;
    private int size = 0;
    private int capacity;
```

```
public class Main {
    public static void main(String[] args) {
        // Step 1: DCS creates an empty student list
        StudentList dcs = new StudentList();
        System.out.println(" DCS created an empty student list.\n");

        // Step 2: Students get admitted into the department
        dcs.addStudent(new Student(1, "Ahmed")); // First admission
        dcs.addStudent(new Student(2, "Ayesha")); // Second admission
        dcs.addStudent(new Student(3, "Bilal")); // Third admission
        System.out.println(" New admissions added to DCS:");
        System.out.println(dcs);

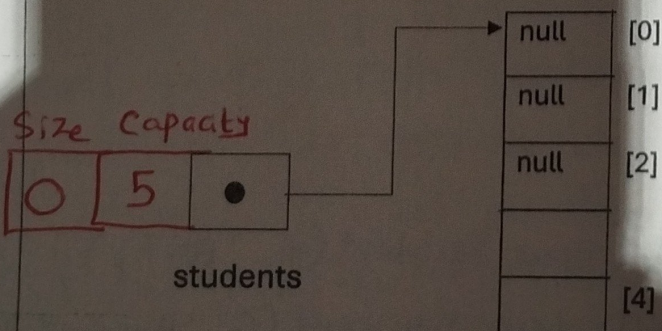
        // Step 3: Another student joins in the middle (special case)
        dcs.addStudent(1, new Student(4, "Fatima"));
        System.out.println(" Fatima joined and was placed at index 1:");
        System.out.println(dcs);
    }
}
```

Step 2: Write null constructor of StudentList.java using given memory intuition

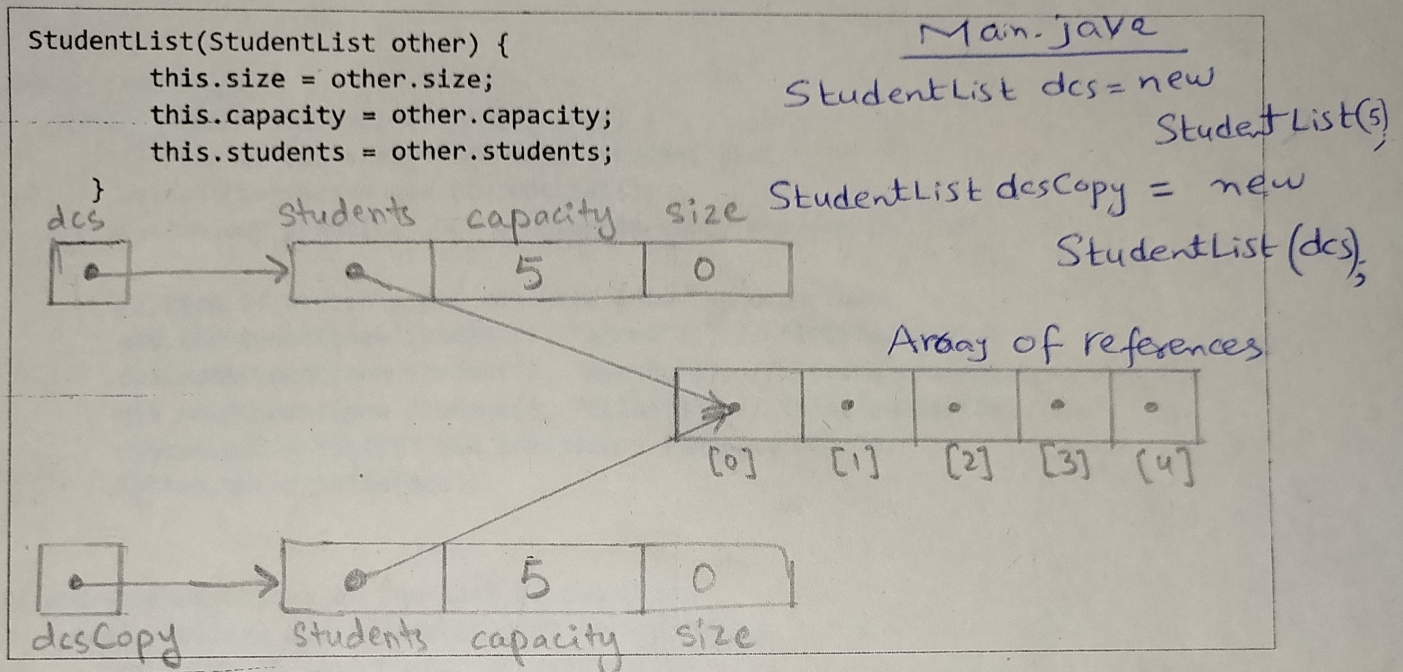
```
StudentList() {
```

```
    this.capacity = 5;
    this.students = new Student[5];
```

```
}
```



Step 3: Draw memory map for copy constructor from class StudentList.java



Step 4: Complete the body of following Constructor of class StudentList.java

Man.java StudentList dcs2 = new StudentList(10, dcs.getStudents());

```
public StudentList(int capacity, Student[] students) {
    this.capacity = capacity;
    this.students = students;
}
```

Note version #03 of getStudents() from week 14

dcs.getStudents() // create deep copy of dcs

Step 5: Can we write constructor of class StudentList.java as follows? Justify your answer

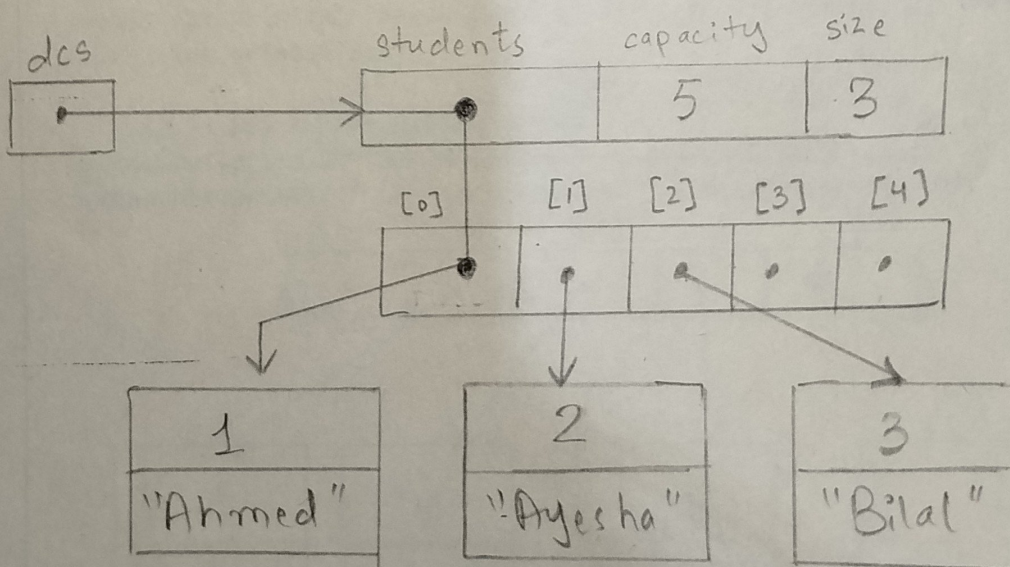
```
public StudentList() {
    this(100, new Student[100]); // array of references
}

public StudentList(int capacity) {
    this(capacity, new Student[capacity]);
}
```

Yes, we can use this(...) in a constructor to call another constructor of the same class. It promotes code reusability and avoids dupli-

Step 6: Draw Memory map for given Main.java

```
public class Main {  
    public static void main(String[] args) {  
        // Step 1: DCS creates an empty student list  
        StudentList dcs = new StudentList();  
        System.out.println(" DCS created an empty student list.\n");  
  
        // Step 2: Students get admitted into the department  
        dcs.addStudent(new Student(1, "Ahmed")); // First admission  
        dcs.addStudent(new Student(2, "Ayesha")); // Second admission  
        dcs.addStudent(new Student(3, "Bilal")); // Third admission  
        System.out.println(" New admissions added to DCS:");  
        System.out.println(dcs);  
    }  
  
    // Add student object at the last of the list  
    public boolean addStudent(Student newStudent) {  
        // Check if seat number already exists  
        for (int i = 0; i < size; i++) {  
            if (students[i].getSeatNo() == newStudent.getSeatNo()) {  
                System.out.println("Error: Seat number already exists!");  
                return false;  
            }  
        }  
        // Expand if full  
        if (size == capacity) {  
            expandCapacity();  
        }  
        // Add student  
        students[size++] = newStudent;  
        return true;  
    }  
}
```



Step 7: complete missing lines and draw Memory map for addStudent at given index

// Lines in Main.java

// Step 3: Another student joins in the middle (special case)

dcs.addStudent(1, new Student(4, "Fatima"));

System.out.println(" Fatima joined and was placed at index 1:");

System.out.println(dcs);

// Lines in StudentList.java

// Add student object at the given index

public boolean addStudent(int idx, Student newStudent) {

if ($idx < 0 \parallel idx > size$) {

System.out.println("Error: Index Out of Bound Exception.");

return false;

}

// Check if seat number already exists

for (int i = 0; i < size; i++) {

if ($students[i].getSeatNo() == newStudent.getSeatNo()$) {

System.out.println("Error: Seat number already exists!");

return false;

}

}

// Expand if full

if ($size == capacity$) {

expandCapacity();

}

// Add student

for (int i = size; i > idx; i--) {

students[i] = students[i - 1];

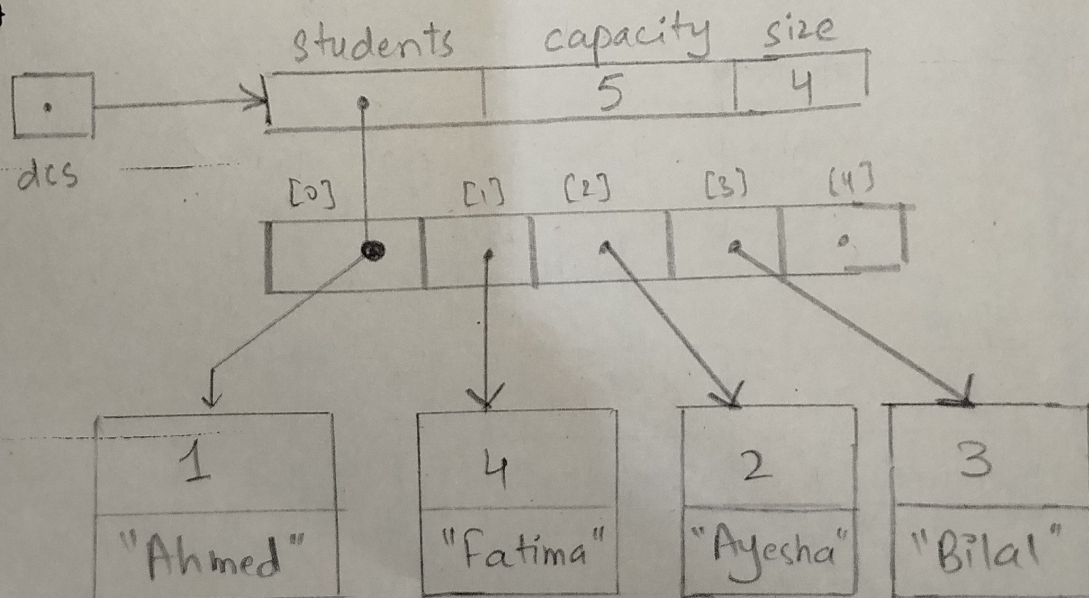
}

students[idx] = newStudent;

size++;

return true;

}



Step 8: Now we need to understand search, and remove the same way.

```
public class Main {
    public static void main(String[] args) {
        // Step 1: DCS creates an empty student list
        StudentList dcs = new StudentList();
        System.out.println(" DCS created an empty student list.\n");

        // Step 2: Students get admitted into the department
        dcs.addStudent(new Student(1, "Ahmed")); // First admission
        dcs.addStudent(new Student(2, "Ayesha")); // Second admission
        dcs.addStudent(new Student(3, "Bilal")); // Third admission
        System.out.println(" New admissions added to DCS:");
        System.out.println(dcs);

        // Step 3: Another student joins in the middle (special case)
        dcs.addStudent(1, new Student(4, "Fatima"));
        System.out.println(" Fatima joined and was placed at index 1:");
        System.out.println(dcs);

        // Step 4: DCS searches for students
        int idxBySeat = dcs.searchStudent(2);
        System.out.println(" Seat number 2 (Ayesha) found at index: " + idxBySeat);

        int idxByName = dcs.searchStudent("Bilal");
        System.out.println(" Bilal found at index: " + idxByName + "\n");

        // Step 5: A student updates record (name correction or replacement)
        dcs.replace(3, new Student(3, "Hina"));
        System.out.println(" Record updated for seat 3 (Bilal replaced by Hina):");
        System.out.println(dcs);

        // Step 6: A student leaves the department
        dcs.removeStudent(1);
        System.out.println(" Ahmed (seat 1) left the department:");
        System.out.println(dcs);

        // Step 7: DCS checks system status
        System.out.println(" Current Size: " + dcs.getSize());
        System.out.println(" Current Capacity: " + dcs.getCapacity());
    }
}
```

Step 9: Lets understand search and replace

// Search student from the list by seat number and returns the index

```
public int searchStudent(int seatNo) {  
    for (int i = 0; i < size; i++) {  
        if (students[i].getSeatNo() == seatNo) {  
            return i;  
        }  
    }  
    return -1;  
}
```

// Search student from the list by name and returns the index

```
public int searchStudent(String name) {  
    for (int i = 0; i < size; i++) {  
        if (students[i].getName().equalsIgnoreCase(name)) {  
            return i;  
        }  
    }  
    return -1;  
}
```

// Replace student at the given seat number

```
public void replace(int seatNo, Student newStudent) {
```

```
    int idx = searchStudent(int seatNo);
```

```
    if (idx == -1) {  
        System.out.println("ERROR: Seat number doesn't exist in the list. ");  
        return;  
    }  
    students[idx] = newStudent;  
}
```

1. What is the output of following program.
Justify Your answer as well briefly.

```
Public class Main {
```

```
    P S V main (String[] args) {
```

```
        // original
```

```
        StudentList list1 = new StudentList(3);
```

```
        Student S1 = new Student("Ali");
```

```
        list1.getStudents()[0] = S1;
```

```
        // copy (shallow)
```

```
        StudentList list2 = new StudentList(list1);
```

```
        list2.getStudents()[0].setName("Sara");
```

```
        S.o.pln(list1.getStudents()[0].getName());
```

```
    }
```

Output :
Sara

2. Which version of getStudents() is more safer and doesn't break Encapsulation. Version #03 assumes class Student has copy Constructor.

Version #01	Version #02	Version #03
<pre>P Student[] getStudents() { return this.Students; }</pre> ↓ reference of array of references ↓ Encapsulation breaks	<pre>P Student[] getStudents() { return Arrays.copyOf(Students, size); }</pre>	<pre>P Student[] getStudents() { Student[] copy = new Student[size]; for (int i=0; i<size; i++) copy[i] = new Student (Students[i]); return copy; }</pre>

Q2(a) Complete the missing code.

```
class StudentList implements Cloneable {  
    private Student[] students;  
    private int size = 0;  
    " " capacity;
```

@Override

protected Object clone() throws CloneNotSupportedException {

StudentList cloned = (StudentList) super.clone();

return cloned;

}

public StudentList deepClone() {

StudentList deepCopy = new StudentList(this.capacity);

deepCopy.size = this.size;

deepCopy.students = this.students;

for(int i=0; i < size, i++) {

deepCopy.students[i] = this.students[i];

}

return deepCopy;

}

// end class

Q2(b) What is the output of following:

```
public class Main {  
    public static void main(String[] args) throws CloneNotSupportedException {  
        StudentList original = new StudentList(3);  
        original.addStudent(new Student(1, "Ali"));  
        " " " " (2, "Sara");  
    }  
}
```

Q2(b) Continued

StudentList shallow = (StudentList) original.clone();

StudentList deep = original.deepClone();

original.removeStudent(1);

" . addStudent(new Student(3, "changed"));

S.o.pln("Original: \n" + original);

" ("Shallow Copy: \n" + shallow);

" ("Deep Copy: \n" + deep);

Q3 Sorting Object Array (import java.util.Arrays)

Assume Class Employee is available to you.

Public class Main {

public static void main(String[] args) {

Employee[] empArr = new Employee[4];

empArr[0] = new Employee(10, "HT", 50, 200000);

empArr[1] = new Employee(27, "NY", 18, 100000);

empArr[2] = new Employee(29, "SM", 19, 105000);

(error) ClassCastException empArr[3] = new Employee(18, "AD", 22, 102500);

Arrays.sort(empArr);

S.o.pln("Default Sorting of Employees list: \n" + Arrays.toString(empArr));

Write & Justify the output

* Not Compatible with Employee datatype (user defined DT)

Q4 Complete the missing code.

```
P S V main(String[] args) {
```

```
Random g = new Random();
```

```
Point[] points = new Point[100];
```

```
for (int i=0; i<100; i++) // create 100 random points  
    points[i] = new Point(g.nextInt(100), g.nextInt(100));
```

```
Arrays.sort(points); // invokes compareTo
```

```
for (int i=0; i<100; i++)  
    s.o.pln(points[i]);
```

```
s.o.pln("\n\n");
```

```
Arrays.sort(points, new Comparator());
```

```
for (int i=0; i<100; i++)  
    s.o.pln(points[i]);
```

```
} // end main.
```

```
private static class Point implements Comparable<Point> {
```

```
// attributes
```

```
// constructors
```

```
// setters and getters
```

```
// toString()
```

```
// add, subtract
```

```
public int compareTo(Object o)
```

```
{ Point A = (Point) o; // Type casting
```

```
if (this.x != A.getX())
```

```
    return -1;
```

```
else
```

```
    return 0;
```

private static class Comparator

implements Comparator<Point> {

public boolean equals(Point P1)

{ return false; }

// note: Comparator does not
store Point.

public int compare(Point P1, Point P2) {

// trick to
compare (diff) if (P1.getY() != P2.getY())
return P1.getY() - P2.getY();

else

return 0;

}

} // end class Comparator

} // end class Point

Q5

Complete following version of compareTo
of class Point.

public class Point implements Comparable<Point> {
private int x;
private int y;

public int compareTo(Point other) {

if (x < other.x) { return -1; }

else if (x > other.x) { return +1; }

else if (y < other.y) { return -1; }

else if (y > other.y) { return +1; }

else 0; }

Q6 Complete &
Write output of following Program.

```

public class Main {
    public static void main (String[] args) {
        StudentList list = new StudentList(5);
        list.addStudent(new Student(3, "Ali"));
        "      "      (1, "Sara");
        "      "      (2, "Bilal");
        "      "      "
        s.o.pln("=== Original List ===");
        s.o.pln(_____);

        // Create a shallow copy for sorting
        Student[] sortedCopy = Arrays.copyOf(
            (list.getStudents(),
            list.getSize());

        // Sort the shallow copy
        by Seat Number (ascending)
        Arrays.sort(sortedCopy, (a,b) -> a.getSeatNo()
            b.getSeatNo());

        s.o.pln("=== Sorted Copy (Shallow Copy) ===");
        for (Student s : sortedCopy) {
            s.o.pln(s);
        }
        s.o.pln("=== Original List After Sorting ===");
        s.o.pln(_____);
    }
}
    
```

why?

Lambda?

for each loop why?

Student
- seatNo: int
- name: String
+ Student()
...

```

class StudentList {
    ...
    public int getSize()
    { return this.size; }
    public Student[] getStudents()
    { return this.students; }
}
    
```

```
public class PolymorphicArray {
    public static void main(String[] args) {
        Truck[] truck = new Truck[2]; // array of Truck refs
        truck[0] = new HGV();
        truck[1] = new Garbage();

        for (int i = 0; i < truck.length; i++) {
            truck[i].load("stuff");
        }
    }
}
```

Question 7 There are two errors in the following code. Describe errors and fix them.

```
// Problem3Fixed.java
interface C {
    public int m();

    public abstract int n();
}

abstract class D implements C {
    @Override
    public int m() {
        return 3;
    }
}

abstract class E extends D {
    @Override
    public int n() {
        return 4;
    }
    // Provide another concrete method o()
    public int o() {
        return 5;
    }
}

class F extends E {
    // no extra code needed; inherits m(), n(), o()
}

public class Problem3Fixed {
    public static void main(String[] args) {
        // 1) Instantiate F (concrete)
        F f = new F();
        System.out.println("f.m() = " + f.m());
        System.out.println("f.n() = " + f.n());
        System.out.println("f.o() = " + f.o());
        // 2) Use E reference to E object
        E e = new E();
        System.out.println("e.m() = " + e.m());
        System.out.println("e.n() = " + e.n());
        System.out.println("e.o() = " + e.o());
        // 3) Use interface type C to refer to F
        C c = new F();
        System.out.println("c.m() = " + c.m());
        System.out.println("c.n() = " + c.n());
        // To call o(), you must ???
        System.out.println("c.o() = " + c.o());
    }
}
```

Question 5**Give Answer/Reasoning/output**

```

class RationalNumber {
    private final int num;
    private final int den;

    public RationalNumber(int n, int d) {
        assert d > 0;
        this.num = n;
        this.den = d;
    }

    @Override
    public boolean equals(Object other) {
        if (other == null)
            return false; // Way 1
        if (!(other instanceof RationalNumber))
            return false; // Way 2
        RationalNumber that = (RationalNumber) other;
        return this.num == that.num && this.den == that.den;
    }
}

public class EqualsPractice2 {
    public static void main(String[] args) {
        RationalNumber r1 = new RationalNumber(10, 5);
        RationalNumber r2 = new RationalNumber(10, 5);

        Object o1 = r1;
        Object o2 = r2;

        System.out.println(r1.equals(r2));
        System.out.println(o1.equals(o2));
        System.out.println(r1.equals(o2));
        System.out.println(o1.equals(r2));
        System.out.println(r1.equals(o1));

        Object o3 = null;
        System.out.println(r1.equals(o3));
    }
}

```

Question 6 Provide clear comments on static and run time compiler behaviour

```

/ PolymorphicArray.java
class Truck {
    public void load(String payload) {
        // default (empty) implementation
    }
}

class Garbage extends Truck {
    @Override
    public void load(String payload) {
        System.out.println("Our garbage truck eats " + payload);
    }
}

class HGV extends Truck { // Heavy Goods Vehicle
    @Override
    public void load(String payload) {
        System.out.println("We are transporting " + payload + " in our Heavy Goods Vehicle.");
    }
}

```

Question 3
 class TestStuff {
 public static v
 Stuff s = n
 System
 do
 s

Question 3

```
class TestStuff {
    public static void main(String[] args) {
        Stuff s = new Stuff("In", 5);
        System.out.println(s);
        double doubleValue = 2.5;
        s.doSomething(doubleValue);
        s = new Stuff("more", 3);
        String str = "words";

        System.out.println(s.changeSomething(str));
        System.out.println(s);
        System.out.println(str);
    }
}
```

```
class Stuff {
    private static final int n = 2;
    private String string;
    private int num;

    public Stuff(String s, int num) {
        this.num = num;
        string = s;
    }

    public void doSomething(double d) {
        d = d - num;
        System.out.println(this);
    }

    public double changeSomething(String s) {
        s = string;
        return num * n;
    }

    public String toString() {
        return string + " has " + num;
    }
}
```

Answer/Reasoning/output**Question 4**

// Version 1: equals(RationalNumber r) only (no overriding)

```
class RationalNumber {
    private final int num;
    private final int den;

    public RationalNumber(int n, int d) {
        assert d > 0;
        this.num = n;
        this.den = d;
    }

    public boolean equals(RationalNumber r) {
        return this.num == r.num && this.den == r.den;
    }
}

public class EqualsPractice1 {
    public static void main(String[] args) {
        RationalNumber r1 = new RationalNumber(10, 5);
        RationalNumber r2 = new RationalNumber(10, 5);
        Object o1 = r1;
        Object o2 = r2;
        System.out.println(r1.equals(r2));
        System.out.println(o1.equals(o2));
        System.out.println(r1.equals(o2));
        System.out.println(o1.equals(r2));
        System.out.println(r1.equals(o1));
    }
}
```

Question 1

```

public class InheritancePractice {
    public static void main(String[] args) {
        SubClass2 obj = new SubClass2();
    }
}

class SubClass2 extends SuperClass2 {
    public SubClass2() {
        super(10);
        System.out.println("This is the subclass constructor.");
    }
}

class SuperClass2 {
    public SuperClass2() {
        System.out.println("This is the superclass no-arg constructor.");
    }
    public SuperClass2(int arg) {
        System.out.println("The following argument was " + arg + " passed to the superclass constructor.");
    }
}

```

Answer/Reasoning/Output/Numbering for s.o.pln**Question 2 (Provide line by line reasoning with memory map and fix the error)**

```

class P {
    static int x = 3;
    int y = 7;
    public static void q(int w) { x = w; }
}

class C extends P {
    static int u = 5;
    public void r(int v) {
        y = v;
        u = 8;
    }
}

class M {
    public static void main(String[] args) {
        P a1 = new P();
        C a2 = new C();
        P a3 = a2;
        a1.r(6);
    }
}

```

Answer/Reasoning/Memory Map/Output



Course Supervisor: Dr. Humera Tariq

Date : 31st May 2022

Course Title: Software Development

Time: 9:00 AM to 11:00 AM

Student Name :

Seat No.:

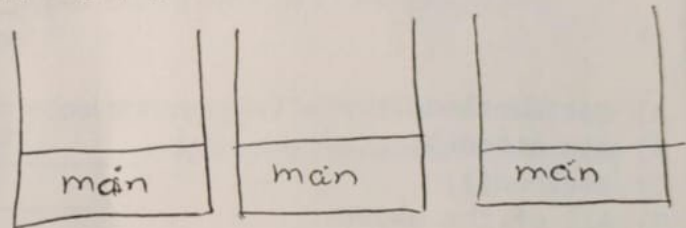
Question 1: Circle the correct answer. Write 1-2-line reasoning. Support your answer with memory diagrams or function call stack. (Q1: 10 Marks)

class Q1a {

(1+1+1 = 3 Marks)

```
public void run(int count){
    System.out.println("I am sad because count is" + count);
    count--;
    if(count > 0)
        run(count);
    System.out.println("I am happy because count is" + count);
}
```

```
public static void main(String[] args) {
    Q1a q1a = new Q1a();
    q1a.run(2);
}
```

Stack Visualization

- A) Compilation Error at Line 6.
 B) Runtime Exception at Line 6.
 C) Prints following Lines:
 I am sad because count is 2
 I am happy because count is 1
 D) Prints
 I am sad because count is 2
 I am sad because count is 1
 I am happy because count is 0
 I am happy because count is 1

class Q1b {

(1+1 = 2 Marks)

```
public static void main(String[] args) {
    double iRand;
    iRand = Math.random();
    System.out.println(iRand*100);
}
```

- A) A random number between 0 and 1.
 B) A random number between 0 and 100.
 C) A random number between 0 and 10.
 D) A compile time error about random being an un-recognized method.

Student Name:

Choose the correct missing statement to invoke amethod() immediately after object creation in main(). Write 1-2-line reasoning. (1+1+1 = 3 Marks)

```
class Q1c {
```

```
    public static void main(String[] args) {
```

```
        Q1c q = new Q1c();
```

```
// I am missing statement
```

```
    }
```

```
    public static void amethod(){
```

```
        int ia[] = new int[4];
```

```
        for (int i=0;i<ia.length;i++){
```

```
            ia[i] = i;
```

```
            System.out.println(ia[i]);
```

```
        }
```

```
    }
```

```
}
```

A) Q1c.amethod();

B) q.amethod();

C) amethod();

D) All of the above.

Given the following code , which of the following if inserted at the comment //Here will allow the code to compile and run without error. Write 1-2 line reasoning. (1+1 = 2 Marks)

```
class Base {}
```

```
class Q1d extends Base {
```

```
    static boolean b1 = false;
```

```
    static int i = -1;
```

```
    static double d =10.1;
```

```
    public static void main(String argv[]) {
```

```
        System.out.println("Hello");
```

```
        Q1d m = new Q1d();
```

```
        Base b = new Base();
```

```
        // Here
```

```
    }
```

```
}
```

A) **b** = m;

B) m = b;

C) **i** = d;

D) b1=i;

Immediately
Marks)

Question 2: Read and study the situations and answer accordingly using given hints/vocabulary from given word bank and UML: **(10 marks)**

Low coupling, Rule of Abstraction, Liskov Substitution Principal(LSP), Open/Closed Rule, The rule of Transparency, Unit Testing, Type Safety, Type Casting, Reference type, interface type

- 1- A Bank object has the variable accounts, which maps an account number to the corresponding BankAccount object. The type of the variable is `HashMap<Integer, BankAccount>`. The rule of _____ suggests that the variable should have the interface type `Map<Integer, BankAccount>` instead.

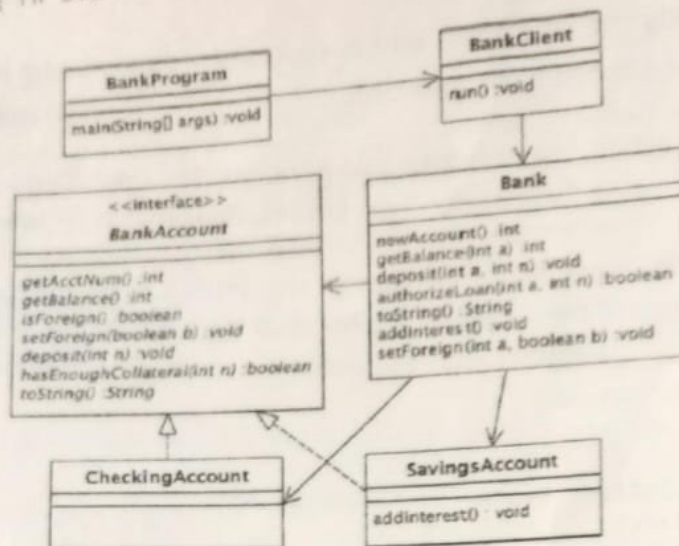
```
public class Bank {  
    private HashMap<Integer, BankAccount> accounts;  
    private int nextacct;  
    public Bank(HashMap<Integer, BankAccount> accounts, int n) {  
        this.accounts = accounts;  
        nextacct = n;  
    } ... }
```

- 2- If a variable is class-typed then it can hold a _____ to any object of that class. If a variable is _____ typed then it can hold a reference to any object whose class implements that interface.
- 3- Name the programming construct/feature/principal that you need to recall as a developer while look at Line 2 of given code: _____. Also state the name of relationship between two classes _____.
- ```
BankAccount ba = new SavingsAccount(1);
SavingsAccount sa = ba;
```
- 4- To the extent possible, a program should be \_\_\_\_\_ for extension but \_\_\_\_\_ for modification.
- 5- If type X extends type Y then an object of type X can always be used wherever an object of type Y is expected. This principal is known as \_\_\_\_\_.
- 6- In the polymorphic version of banking application, the classes **CheckingAccount** and **SavingsAccount** both implement the **BankAccount interface**. The `addInterest()` method iterates through all the accounts but adds interest only to the savings accounts. Since the elements of the variable accounts are of type BankAccount, and BankAccount does not have an `addInterest()` method, we need to ensure type safety. Study the following snippet of code and complete the statement on right side using appropriate object type and exception.

```
public void addInterest() {
 for (BankAccount ba : accounts.values()) {
 SavingsAccount sa = (SavingsAccount) ba;
 sa.addInterest(); } }
```

if **ba** ever refers to a \_\_\_\_\_,  
type of object then the typecast will throw  
a \_\_\_\_\_ at runtime.

Overwriting in blanks is strictly prohibited.



Will the following code compile(Yes or No)? Write 2-3-line justification .

```

public void addInterest() {
 for (BankAccount ba : accounts.values())
 if (ba instanceof SavingsAccount)
 ba.addInterest();
}

```

}

The software development principal that can reduce coupling between classes is Interface Segregation. If you apply this rule to addInterest() code in part(7) above, what advantage you will observe. Write 2-3-line justification to prove that you understand the application of this rule.

- Study UML and tell which of the lines are legal or illegal. Write reason for illegal line only.

```

SavingsAccount sa = new SavingsAccount(1);
BankAccount ba = new SavingsAccount(2);

```

|                  |                   |
|------------------|-------------------|
| sa.deposit(100); | sa.addInterest(); |
| ba.deposit(100); | ba.addInterest(); |

0- Complete given code to find the largest account by calling findMax(), which performs a linear search of the list. It initializes the current maximum to be the first element.

```

private static SavingsAccount findMax(ArrayList<SavingsAccount> a) {
 SavingsAccount max = a.get(0);
 for (int i=1; i<a.size(); i++) {
 if (_____)
 max = _____;
 }
 return max;
}

```

### Question 3: Read and study the question and answer accordingly: (10 marks)

Which piece of code is efficient to design and access hash map entries and why? Write 3-4 line justification.

**Listing 1**

```
HashMap<String,Integer> m = new HashMap<>();
m.put("a", 1);
m.put("b", 4);
int x = m.get("a");

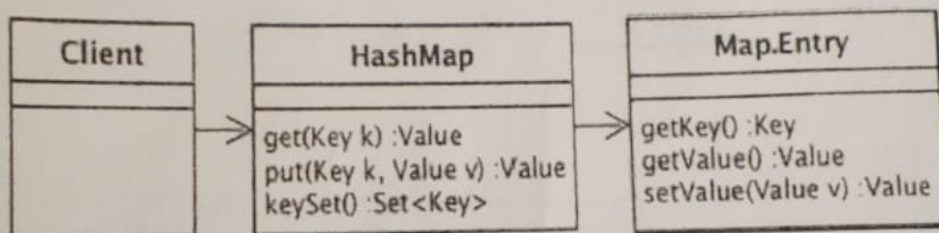
Set<Map.Entry<String,Integer>> entries = m.entrySet();
for (Map.Entry<String,Integer> e : entries) {
 String s = e.getKey();
 int y = e.getValue();
 System.out.println(s + " " + y);
}
```

**Listing 2**

```
HashMap<String,Integer> m = new HashMap<>();
m.put("a", 1);
m.put("b", 4);
int x = m.get("a");

Set<String> keys = m.keySet();
for(String s: keys) {
 int y = m.get(s);
 System.out.println(s + " " + y);
}
```

2- What single change you will do in following UML diagram so that it matches Listing 1 in Q3(part 1). Please do write your change clearly for full credits.



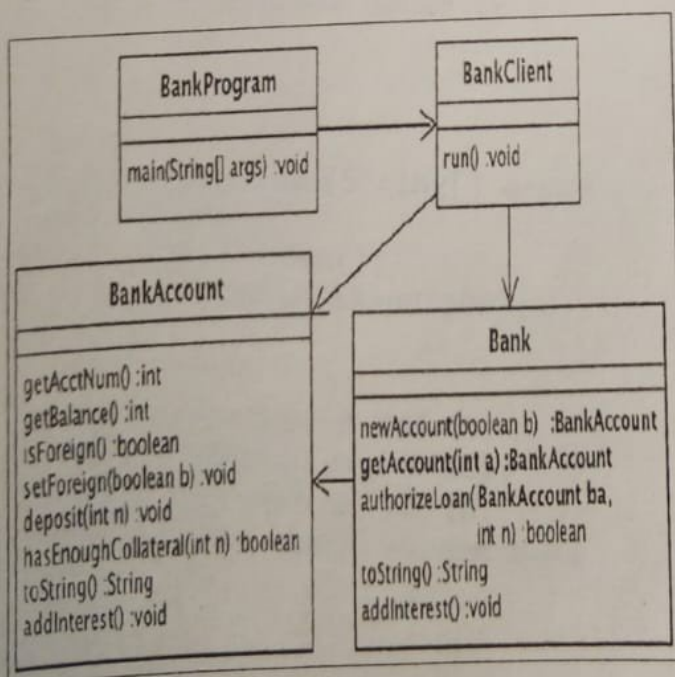
3- Circle the two rules which often conflict with each other ?

a) Single Responsibility Rule b) Encapsulation c) Most qualified class d) Low coupling

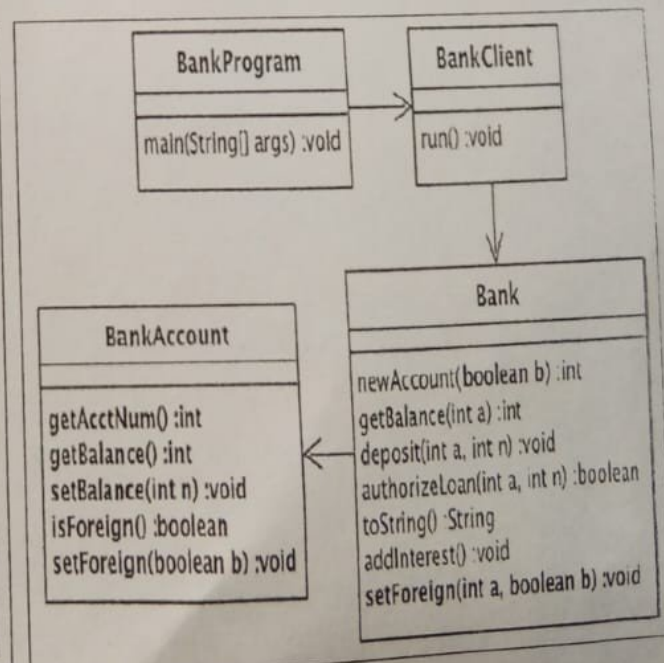
4- Renaming a class, method, or variable; changing the implementation of a variable from one data type to another; and splitting a class into two; all are examples of \_\_\_\_\_.

a) modularity b) usecases c) command processing d) refactoring e) accessors & mutators

5- Which design is better and why? Write 3-4-line justification using development rules.



Design I



Design II